

PROJECT REPORT

On

Time Capsule For Digital Legacy

Submitted to Rajasthan Technical University
in partial fulfillment of the requirement for the award of the degree of

B.TECH.

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted By

Dev Choudhary(PIET22CA016)

Rishank Jain(PIET22CA043)

Aayush Pareek(PIET22CA001)

Under the Guidance of

Dr. Ajay Maurya

at



POORNIMA INSTITUTE OF ENGINEERING & TECHNOLOGY, JAIPUR

RAJASTHAN TECHNICAL UNIVERSITY, KOTA

April, 2026

CERTIFICATE

This is to certify that the project entitled **“Time Capsule For Digital Legacy”** has been submitted for the Bachelor of Computer Engineering, Poornima Institute of Engineering and Technology, Jaipur during the academic year 2021–2022. It is a bonafide piece of project work carried out by **Dev Choudhary, Rishank Jain and Aayush Pareek** towards the partial fulfillment for the award of the Degree (B.Tech.) under the guidance of **“Dr Ajay Maurya”**. No part thereof has been submitted by them for any degree or diploma.

Project Guide: Mr./Mrs./Dr. Alka Rani (Assistant Professor)

Project Coordinator: Dr. Ajay Maurya

Head of Department: Prof. (Dr.) Budesh Kanwar (H.O.D. AI & DS)

CANDIDATE’S DECLARATION

We, **Dev Choudhary (PIET22CA016)**, **Rishank Jain (PIET22CA043)** and **Aayush Pareek (PIET22CA001)** B.Tech (Semester–VIII) of Poornima Institute of Engineering and Technology, Jaipur, hereby declare that the Project Report entitled “**Time Capsule For Digital Legacy**” is an original work and the data provided in the study is authentic to the best of our knowledge. This report has not been submitted to any other institute for the award of any other degree.

Signatures:

(PIET22CA016)

(PIET22CA043)

(PIET22CA001)

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people who directly or indirectly contributed to the development of this work and who influenced our thinking, behavior, and acts during the course of study.

We express our sincere gratitude to Prof. (Dr.) Dinesh Goyal, Director, PIET for providing us an opportunity to undergo this Major Project as part of the curriculum.

We are thankful to Prof. (Dr.) Budesh Kanwar, HOD, AI & DS for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence, and blessings.

We are thankful to Dr. Ajay Maurya for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence, and blessings.

We also extend our sincere appreciation to Ms. Alka Rani who provided her valuable suggestions and precious time in accomplishing our Project Report.

Lastly, we would like to thank the Almighty and our parents for their moral support and friends with whom we shared our day-to-day experience and received lots of suggestions that improved our quality of work.

Signatures:

(PIET22CA016)

(PIET22CA043)

(PIET22CA001)

TABLE OF CONTENTS

CHAPTER NO.	TOPICS	PAGE NO.
	TITLE PAGE	I
	CERTIFICATE	II
	CANDIDATE'S DECLARATION	III
	ACKNOWLEDGEMENT	IV
	TABLE OF CONTENTS	V
	LIST OF FIGURES	VI
	LIST OF TABLES	VII
	ABSTRACT	VIII
1	INTRODUCTION Project Aim and Objective Problem Statement Software Requirements Hardware Requirements	1
2	LITERATURE SURVEY	
3	PROJECT MANAGEMENT Project Integration Management Project Scope Management Project Time Management Project Cost Management Project Quality Management Project Human Resource Management Project Communication Management Project Risk Management Project Procurement Management Project Management Tools	
4	TECHNOLOGY APPLIED Agile project management and Scrum Core values of agile Principles of agile Steps in the agile methodology	

CHAPTER NO.	TOPICS	PAGE NO.
	POs and their relevance to project	
5	PRODUCT BACKLOG DESIGN Product Backlog Sprint Backlog-1 Sprint Backlog-2 Sprint Backlog-3 Sprint Backlog-4	
6	PROJECT IMPLEMENTATION Sprint Backlog-1 Sprint Backlog-2 Sprint Backlog-3 Sprint Backlog-4	
7	RESULTS Outcome Benefits to society Future Scope	
	REFERENCES	

List of Figures

5.1	Sprint 1 Screenshots	23
5.2	Sprint 1 Burn Down Chart	23
5.3	Sprint 2 Screenshots	24
5.4	Sprint 2 Burn Down Chart	24
5.5	Sprint 3 Screenshots	25
5.6	Sprint 3 Burn Down Chart	25
5.7	Sprint 4 Screenshots	26
5.8	Sprint 4 Burn Down Chart	26
6.1	Entity–Relationship Diagram of Time Capsule for Digital Legacy	29
6.2	Code Snippet 1: Backend Implementation	29
6.3	Code Snippet 2: Encryption Logic	30
6.4	Code Snippet 3: Scheduler Module	30
6.5	Application Home Screen	31
6.6	Time Capsule Creation Screen	31

List of Tables

5.1	Product Backlog for Time Capsule for Digital Legacy	22
5.2	Sprint 1 Backlog	23
5.3	Sprint 2 Backlog	23
5.4	Sprint 3 Backlog	24
5.5	Sprint 4 Backlog	25

ABSTRACT

The rapid expansion of digital technologies has resulted in individuals accumulating large amounts of personal data across multiple online platforms, devices, and cloud services. As people increasingly rely on digital storage, the question of how this information can be preserved, accessed, or passed on after one's lifetime has become critical. This project, **“Time Capsule for Digital Legacy,”** presents a system designed to store, secure, and preserve personal digital assets for future retrieval.

The proposed system allows users to upload important documents, media, memories, and personal messages into a secure digital repository that can be accessed at a predetermined future time or by authorized individuals. The architecture incorporates **encryption, authentication mechanisms, and automated release triggers** to ensure confidentiality, integrity, and controlled access. By combining secure cloud storage with user-centric access policies, the system provides a reliable way to maintain a long-term digital legacy.

This project aims to address the growing need for structured digital preservation by offering a platform that is secure, scalable, and user-friendly. It ensures that valuable digital content remains protected and retrievable—acting as a modern “time capsule” for memories, personal information, and identity.

Keywords/Abbreviations: Digital Legacy, Time Capsule, Digital Preservation, Secure Storage, Authentication, Encryption, Cloud Repository

1. INTRODUCTION

In the modern digital era, individuals generate an enormous volume of personal data throughout their lifetime. This data includes photographs, videos, documents, social media content, passwords, financial records, and personal messages distributed across phones, laptops, cloud storages, and online platforms. As people increasingly rely on digital mediums to store memories and essential records, the concept of preserving this digital information for future generations has become increasingly relevant.

Traditionally, physical time capsules were used to preserve objects or written messages for the future. However, with the shift toward digital lifestyles, the need for a secure and organized digital equivalent has emerged. The project titled **“Time Capsule for Digital Legacy”** aims to solve this necessity by creating a system that stores an individual’s personal and meaningful digital assets and ensures their availability at a predetermined time or for designated beneficiaries.

The digital legacy concept revolves around the idea that a person’s data can outlive them and should remain accessible, secure, and meaningful. Yet, there are numerous challenges associated with long-term digital preservation such as security concerns, data loss, unauthorized access, password mismanagement, and platform shutdowns. Many individuals also lack a structured method to store their memories or confidential information safely for future use.

The proposed system provides a controlled environment where users can upload and manage their digital content in an organized way. It improves upon scattered storage systems by centralizing the process of preservation, adding layers of encryption, access control, and automated release mechanisms. The solution not only ensures data security but also creates a personalized digital inheritance system.

Furthermore, the rapid growth of cyber threats, data breaches, and account hijacking highlights the need for a reliable platform where valuable personal information can be secured. This project addresses these concerns by integrating authentication, encryption, and trigger-based release workflows, making it suitable for long-term preservation.

Overall, the system helps bridge the gap between modern digital life and future digital inheritance by enabling users to store memories, secrets, documents, and personal messages with complete safety and accessibility. It provides peace of mind by ensuring that treasured digital information remains intact and available when needed, whether for personal reflection or as a legacy for the

next generation.

Project Aim and Objective

Aim: To develop a secure digital platform that enables users to store personal data, memories, and confidential information safely and retrieve it at a future time or allow authorized individuals to access it as part of a digital legacy.

Objectives:

- To create a centralized digital repository for long-term preservation of personal data.
- To implement encryption and authentication mechanisms ensuring data safety.
- To provide time-based or user-authorized access to stored digital assets.
- To build a user-friendly interface for uploading, viewing, and managing digital content.
- To ensure privacy, integrity, and controlled access to sensitive information.
- To support storage of multimedia files, documents, and personal messages.
- To prevent unauthorized access through multi-layered security mechanisms.

Problem Statement

As people rely heavily on digital devices and cloud platforms, their personal data becomes scattered, unorganized, and vulnerable to loss or misuse. Users accumulate thousands of digital assets—photographs, important documents, private messages, financial details, and sentimental content—across various platforms. Over time, this information becomes difficult to manage, easy to forget, and potentially inaccessible due to forgotten passwords, device failures, service shutdowns, or cyber-attacks.

Furthermore, after an individual's lifetime, their digital presence is often left unmanaged, leaving families without access to important information and cherished memories. Traditional methods do not support structured digital preservation, and existing cloud storage services lack time-delayed or legacy-based access functionality. There is also no unified solution that allows users to store their digital identity securely for future retrieval.

In addition, concerns related to privacy, confidentiality, unauthorized access, and data breaches further complicate the long-term storage of sensitive information. Without a dedicated system, digital assets remain at risk of being lost, deleted, or compromised.

Thus, there is a strong need for a secure, reliable, and user-friendly digital time capsule system that ensures safe preservation, controlled access, and long-term availability of personal digital content. The challenge lies in building a platform that:

- protects data from unauthorized access,
- ensures confidentiality through encryption,
- allows timed or authorized release of information,
- enables users to organize their digital legacy efficiently,
- remains accessible even after long periods.

This project addresses these issues by developing a secure digital legacy system that acts as a modern time capsule for storing memories, documents, and sensitive information, accessible only at the right time or by the right person.

Software Requirements

- Operating System: Windows / Linux / macOS
- Frontend: HTML, CSS, JavaScript, React
- Backend: Python (Flask)
- Database: PostgreSQL
- Cloud Storage: AWS S3 / Google Cloud Storage / Firebase Storage
- Development Tools: VS Code, Git, Postman
- Security Libraries: JWT Authentication, Encryption APIs

Hardware Requirements

- Processor: Dual-core or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 20 GB free space
- Stable internet connection (for cloud synchronization)
- Smartphone or PC for accessing the platform

2. LITERATURE SURVEY

The rapid digitization of personal information has led to the emergence of the concept of a **digital legacy**, which refers to the digital data, online accounts, and stored memories that a person leaves behind. With the increasing reliance on digital platforms for communication, documentation, and media storage, researchers have highlighted the need for structured and secure digital preservation systems. This chapter presents a comprehensive literature survey on digital legacy management, digital preservation techniques, time-lock mechanisms, and data security models relevant to the proposed system.

Digital Legacy Preservation

Digital legacy management has gained academic interest due to the growth of social media and online identity systems. Studies show that individuals generate vast quantities of digital assets—emails, photos, videos, documents, and social media posts—that require systematic preservation. Researchers such as Odom et al. emphasize that digital possessions hold emotional and cultural value, similar to physical keepsakes. However, traditional cloud storage platforms lack features such as posthumous access control, inheritance mechanisms, or timed release of digital assets.

Social media platforms like Facebook and Google provide limited post-death account management options, such as legacy contacts or inactive account managers. While these features help retain certain information, they are not designed for long-term preservation or controlled future release. Research indicates that users need a platform where digital content can be preserved intentionally and securely, not merely archived by default.

Digital Time Capsules

The concept of a “time capsule” has transitioned from physical objects to digital media. Early digital time capsule projects, such as “FutureMe” and “Timecave,” allowed users to send delayed emails to their future selves. However, these systems provide minimal security, limited storage, and no structured data management. Other platforms like “CapsureDS” and “SafeBeyond” focus

on legacy messaging but rely heavily on manual triggers and lack robust encryption methods. Academic literature highlights the need for digital time capsules that incorporate:

- secure long-term data storage,
- encrypted access control mechanisms,
- automated time-based retrieval systems,
- multi-format data handling,
- flexible inheritance models.

These requirements directly influence the design of modern digital legacy systems.

Data Security and Encryption Techniques

Secure storage of personal information is a widely explored research domain. Various encryption algorithms such as AES, RSA, and ECC are commonly used to protect stored data. Studies have shown that end-to-end encryption enhances confidentiality by ensuring that only authorized users can access stored content. Authentication mechanisms—including password-based, biometric, and two-factor authentication—are crucial for preventing unauthorized access.

A significant contribution in this area is the zero-knowledge encryption model, which ensures that service providers cannot access user data even if storage servers are compromised. Literature also emphasizes the importance of secure key management and user-friendly authentication methods.

Cloud Storage and Scalability

Cloud technologies play a central role in digital legacy systems due to their scalability, redundancy, and availability. Research on cloud-based storage highlights features such as distributed file systems, synchronized backups, and fault-tolerant architectures. Systems like Amazon S3, Google Cloud Storage, and Firebase provide the backbone for long-term digital preservation.

Studies underscore challenges in long-term cloud storage:

- potential service discontinuation,
- data migration issues,
- risks of vendor lock-in,

- long-term key preservation.

These challenges must be addressed to ensure the viability of digital time capsules over decades.

Gap Analysis

Despite advancements in digital preservation and security technologies, current systems lack a unified solution that integrates:

- long-term secure digital storage,
- encrypted multi-format data preservation,
- time-based or inheritance-based access,
- automated release triggers,
- user-centric legacy planning.

Existing platforms are either too simplistic (delayed email services), too restrictive (social media legacy settings), or insufficiently secure (unencrypted legacy messaging services). Academic literature indicates a strong need for a dedicated system that blends encryption, cloud storage, authentication, and automated release workflows.

Conclusion

The literature highlights increasing global concern about digital legacy management and the scarcity of comprehensive systems tailored for long-term preservation. The proposed project, **Time Capsule for Digital Legacy**, aims to fill this gap by incorporating secure storage, access control, and timed-release mechanisms. By synthesizing knowledge from digital preservation research, encryption technologies, and cloud scalability studies, the project aspires to deliver an effective, secure, and future-proof solution for managing and preserving digital assets.

3. PROJECT MANAGEMENT

Effective project management is essential for the successful development, implementation, and deployment of the **Time Capsule for Digital Legacy** system. This chapter elaborates on the various project management domains including integration, scope, cost, time, quality, human resources, communication, risk, procurement, and tools used in the project lifecycle. These practices ensure that the project is executed systematically, meeting the goals, deadlines, and quality benchmarks.

Project Integration Management

Project integration management ensures that all components of the project—planning, execution, monitoring, and closure—function cohesively. For this project, integration involved defining the project charter, developing the project management plan, coordinating development activities between team members, and ensuring smooth integration of frontend, backend, and cloud storage functionalities. Regular review meetings were conducted to align tasks with objectives and ensure that module integration, such as authentication, file upload, encryption, and timed release mechanisms, worked coherently.

Project Scope Management

Scope management defines what is included and excluded from the project. The scope of the *Time Capsule for Digital Legacy* system includes user authentication, secure storage, data encryption, time-based release triggers, user-friendly interface, and cloud backups. Out-of-scope elements include biometric authentication, AI-based data classification, and blockchain-based immutability (though these may be considered in future work). A structured scope statement and Work Breakdown Structure (WBS) were created to clearly outline deliverables and prevent scope creep during development.

Project Time Management

Time management is concerned with scheduling activities, estimating durations, and creating a timeline. The project followed a phased development model:

- Requirement gathering – 1 week
- System design and architecture – 2 weeks
- Frontend development – 2 weeks
- Backend API development – 2 weeks
- Encryption and authentication module – 1 week
- Cloud integration – 1 week
- Testing and debugging – 2 weeks
- Documentation and final deployment – 1 week

Gantt charts and task scheduling tools were used to monitor progress. Critical Path Method (CPM) helped identify tasks that directly influenced the project's timeline.

Project Cost Management

Cost management ensures that the project is completed within the approved budget. The project incurred costs primarily in cloud storage services, software tools, internet usage, and testing infrastructure. Most development tools such as VS Code, GitHub, and Firebase (free tier) were cost-efficient, helping reduce overall expenses. Cost estimation techniques such as analogous estimation and resource-based costing were applied to predict and manage expenses. Continuous monitoring ensured that the project remained financially feasible.

Project Quality Management

Quality management focuses on maintaining standards throughout the project lifecycle. Quality requirements included:

- secure encryption for stored data,
- error-free and optimized code,

- intuitive and responsive UI,
- secure authentication mechanisms,
- high availability and reliability of cloud storage.

Testing methods such as unit testing, integration testing, security testing, and usability testing were performed. Validation ensured that the system met user expectations and complied with security best practices.

Project Human Resource Management

This component deals with organizing and managing the project team. The team consisted of:

- **Dev Choudhary** – Backend and cloud integration
- **Rishank Jain** – Frontend development and UI design
- **Aayush Pareek** – Database and security module implementation

Roles were assigned based on skill sets to ensure maximum team efficiency. Collaboration was facilitated using GitHub version control, regular team meetings, and coordinated task allocation.

Project Communication Management

Effective communication played a critical role in the successful execution of the project. Communication channels included WhatsApp groups, GitHub issues, and weekly review meetings with the project guide. Documentation such as progress reports, technical notes, and requirement specifications ensured clarity and transparency throughout the development cycle.

Project Risk Management

Risk management involves identifying, analyzing, and mitigating potential risks. Key risks identified in this project include:

- **Data Security Breaches:** Mitigated through encryption and secure authentication.
- **Cloud Storage Failure:** Addressed via redundant backups.
- **Feature Delays:** Managed by scheduling buffer time.

- **Integration Issues:** Reduced using frequent testing.

Risk analysis was conducted using a probability–impact matrix to prioritize threats and plan mitigation strategies.

Project Procurement Management

Procurement management involves acquiring external resources necessary for the project. Most tools used were open-source; however, procurement included cloud storage services (such as Firebase or AWS), SSL certificates, and testing devices. Vendor comparison and budget evaluation ensured cost-effective procurement.

Project Management Tools

Several software tools were used to plan, monitor, and manage the project efficiently:

- **Trello / Jira:** Task management and sprint planning.
- **Gantt Charts:** Scheduling and progress tracking.
- **Git and GitHub:** Version control and team collaboration.
- **Draw.io / Lucidchart:** Designing UML and architecture diagrams.
- **Postman:** API testing.
- **Firebase Console / AWS Panel:** Cloud service monitoring.

Each tool contributed to better organization, communication, and quality assurance within the project.

4. TECHNOLOGY APPLIED

Agile Project Management and the Scrum framework were applied during the development of the **Time Capsule for Digital Legacy** system to ensure flexibility, iterative progress, continuous feedback, and high-quality deliverables. Agile allowed the team to adapt to changes, refine requirements, and incrementally build a secure and user-friendly application. This chapter explains Agile values, principles, methodology steps, and the role of Product Owners (POs) in relation to the project.

Agile Project Management and Scrum

Agile is an iterative and adaptive project management approach widely used in modern software development. Unlike traditional models such as Waterfall, Agile focuses on continuous improvement, customer involvement, and delivery of incremental functional components rather than a single final release. Scrum, a subset of Agile, organizes work into short development cycles called sprints, enabling regular reviews and rapid adaptability.

Core Values of Agile

The Agile Manifesto defines four core values that guide Agile development:

- **Individuals and interactions over processes and tools:** Prioritizing effective team communication and collaboration.
- **Working software over comprehensive documentation:** Delivering functional increments frequently rather than relying solely on documentation.
- **Customer collaboration over contract negotiation:** Keeping stakeholders involved throughout development to refine requirements.
- **Responding to change over following a plan:** Allowing the project to evolve based on new insights and changing user needs.

These values were essential during the development of the digital legacy system, enabling continuous refinement of security features, cloud integration, and UI components based on feedback.

Principles of Agile

Agile has twelve guiding principles, many of which aligned directly with the goals of this project:

- Deliver valuable software frequently through short iterations.
- Welcome changing requirements, even late in development.
- Emphasize collaboration between developers and stakeholders.
- Build projects around motivated individuals with autonomy.
- Use face-to-face (or real-time digital) communication effectively.
- Working software is the primary measure of progress.
- Maintain a sustainable development pace.
- Promote technical excellence and good design.
- Keep the system simple and avoid unnecessary complexity.
- Use regular reflection meetings to improve processes.

These principles ensured that the system remained secure, scalable, and user-friendly while allowing flexibility to enhance encryption and authentication features during development.

Steps in the Agile Methodology

The Agile process followed during this project consisted of the following key steps:

1. **Requirement Gathering:** Initial discussions to understand user needs—secure storage, timed release, encryption, etc.
2. **Backlog Creation:** All requirements were converted into user stories and placed in the product backlog.
3. **Sprint Planning:** The team selected tasks for each 1–2 week sprint based on priority and workload.

4. **Sprint Execution:** Development of modules such as authentication, file encryption, cloud upload, and frontend UI.
5. **Daily Stand-ups:** Short team meetings to track progress, identify blockers, and update task status.
6. **Sprint Review:** Demonstration of completed features—secure uploads, login system, cloud storage integration.
7. **Sprint Retrospective:** Discussion of what went well, what could improve, and process adjustments for the next sprint.
8. **Increment Release:** After each sprint, a functional increment of the system was delivered for testing.

This iterative approach allowed the team to make continuous improvements and finalize the system without major delays or rework.

POs and Their Relevance to the Project

In Scrum, the **Product Owner (PO)** is responsible for defining product vision, setting priorities, and ensuring the developed features align with user requirements.

For the **Time Capsule for Digital Legacy** project, the PO role was relevant in the following ways:

- **Requirement Definition:** The PO clarified essential features such as time-based release, encryption, and cloud storage.
- **Backlog Management:** Organizing user stories—secure login, file upload, authentication, timed access.
- **Prioritization:** Security-related tasks were given high priority due to the sensitive nature of user data.
- **User Perspective:** Ensuring the system met the expectations of real users wanting to store personal digital assets safely.
- **Approval of Features:** Completed modules were reviewed and approved before integration.

5. PRODUCT BACKLOG DESIGN

Product Backlog

The product backlog is a prioritized list of user stories and system requirements that represent the entire scope of the **Time Capsule for Digital Legacy** system. It contains high-level functionalities contributed by users, admins, and the development team. Each backlog item includes the user role, requirement, purpose, priority, responsible team, estimated completion date, and relevant remarks.

The following table represents the product backlog extracted from the Excel sheet:

US ID	AS A/AN	I WANT TO	SO THAT	Priority	Responsible	Estimate Date	Remarks
SB1/US1	Frontend Team	Implement User Interface	So that the team can benefit from GUI improvements	3	Frontend Team	2026-01-02	Storage
SB1/US2	Admin	Implement Schedule Delivery	So that the Admin can benefit from scheduling features	3	Admin	2025-09-22	Storage
SB1/US3	User	Implement Encryption Implementation	So that users can benefit from secure access control	3	User	2025-10-06	Compliance

Table 5.1: Product Backlog for Time Capsule for Digital Legacy

Sprint Backlogs

Sprint backlogs are the subset of product backlog items selected for each sprint cycle. Each sprint contains a defined scope, work tasks, responsibilities, and expected deliverables.

Below is the detailed sprint-wise breakdown:

Sprint 1

Introduction: Sprint 1 focused on developing the frontend structure and user interface components. The primary goal was to create the initial screens, user flow, and the basic UI layout to ensure smooth navigation. The sprint also included collaboration between design and development teams to finalize the visual theme, color palette, and layout structure.

Screenshots:

Burn Down Chart:

US ID	I WANT TO	Responsible	Estimate Date
SB1/US1	Implement User Interface	Frontend Team	2026-01-02

Table 5.2: Sprint 1 Backlog

SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T1	[Pat 1] Requirement analysis for: User Story 7: Core Platform & Authentication Feature 7	Dev Choudhary	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T2	[Pat 1] Design architecture for: User Story 7: Core Platform & Authentication Feature 7	Dev Choudhary	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T3	[Pat 1] Create API contract for: User Story 7: Core Platform & Authentication Feature 7	Aayush Pareek	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T4	[Pat 1] Implement backend logic for: User Story 7: Core Platform & Authentication Feature 7	Rishank Jain	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T5	[Pat 1] Write unit tests for: User Story 7: Core Platform & Authentication Feature 7	Aayush Pareek	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T6	[Pat 1] Implement database integration for: User Story 7: Core Platform & Authentication Feature 7	Dev Choudhary	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T7	[Pat 1] Develop frontend component for: User Story 7: Core Platform & Authentication Feature 7	Rishank Jain	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T8	[Pat 1] Integrate frontend with backend for: User Story 7: Core Platform & Authentication Feature 7	Rishank Jain	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T9	[Pat 1] Manual testing & bug fixes for: User Story 7: Core Platform & Authentication Feature 7	Rishank Jain	COMPLETED
SB1/US7	User Story 7: Core Platform & Authentication Feature 7	SB1/US7/T10	[Pat 1] Documentation for: User Story 7: Core Platform & Authentication Feature 7	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T1	[Pat 1] Requirement analysis for: User Story 8: Core Platform & Authentication Feature 8	Dev Choudhary	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T2	[Pat 1] Design architecture for: User Story 8: Core Platform & Authentication Feature 8	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T3	[Pat 1] Create API contract for: User Story 8: Core Platform & Authentication Feature 8	Dev Choudhary	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T4	[Pat 1] Implement backend logic for: User Story 8: Core Platform & Authentication Feature 8	Dev Choudhary	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T5	[Pat 1] Write unit tests for: User Story 8: Core Platform & Authentication Feature 8	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T6	[Pat 1] Implement database integration for: User Story 8: Core Platform & Authentication Feature 8	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T7	[Pat 1] Develop frontend component for: User Story 8: Core Platform & Authentication Feature 8	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T8	[Pat 1] Integrate frontend with backend for: User Story 8: Core Platform & Authentication Feature 8	Aayush Pareek	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T9	[Pat 1] Manual testing & bug fixes for: User Story 8: Core Platform & Authentication Feature 8	Rishank Jain	COMPLETED
SB1/US8	User Story 8: Core Platform & Authentication Feature 8	SB1/US8/T10	[Pat 1] Documentation for: User Story 8: Core Platform & Authentication Feature 8	Aayush Pareek	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T1	[Pat 1] Requirement analysis for: User Story 9: Core Platform & Authentication Feature 9	Dev Choudhary	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T2	[Pat 1] Design architecture for: User Story 9: Core Platform & Authentication Feature 9	Aayush Pareek	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T3	[Pat 1] Create API contract for: User Story 9: Core Platform & Authentication Feature 9	Aayush Pareek	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T4	[Pat 1] Implement backend logic for: User Story 9: Core Platform & Authentication Feature 9	Rishank Jain	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T5	[Pat 1] Write unit tests for: User Story 9: Core Platform & Authentication Feature 9	Dev Choudhary	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T6	[Pat 1] Implement database integration for: User Story 9: Core Platform & Authentication Feature 9	Aayush Pareek	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T7	[Pat 1] Develop frontend component for: User Story 9: Core Platform & Authentication Feature 9	Rishank Jain	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T8	[Pat 1] Integrate frontend with backend for: User Story 9: Core Platform & Authentication Feature 9	Dev Choudhary	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T9	[Pat 1] Manual testing & bug fixes for: User Story 9: Core Platform & Authentication Feature 9	Dev Choudhary	COMPLETED
SB1/US9	User Story 9: Core Platform & Authentication Feature 9	SB1/US9/T10	[Pat 1] Documentation for: User Story 9: Core Platform & Authentication Feature 9	Rishank Jain	COMPLETED
SB1/US10	User Story 10: Core Platform & Authentication Feature 10	SB1/US10/T1	[Pat 1] Requirement analysis for: User Story 10: Core Platform & Authentication Feature 10	Aayush Pareek	COMPLETED

Figure 5.1: Sprint 1 Screenshots

Burndown Chart - Part 1

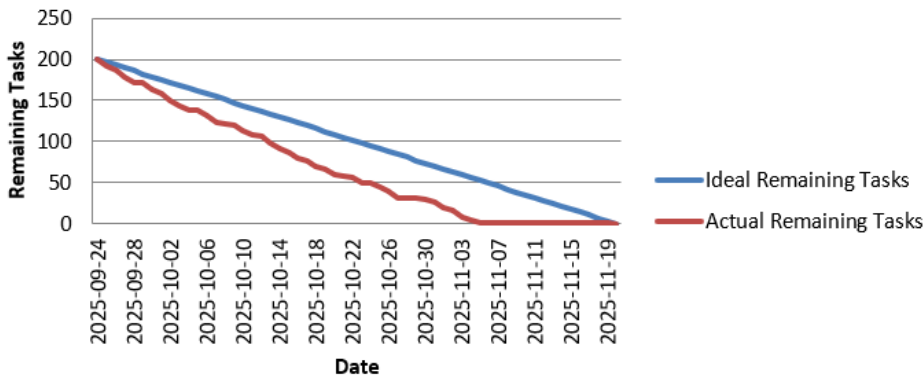


Figure 5.2: Sprint 1 Burn Down Chart

Sprint 2

Introduction: Sprint 2 involved backend administrator features, particularly focused on integrating scheduled delivery functionality. This allowed users and administrators to set future timestamps for releasing stored digital assets. The sprint required coordination with backend and cloud services for data handling and automation.

US ID	I WANT TO	Responsible	Estimate Date
SB1/US2	Implement Schedule Delivery	Admin	2025-09-22

Table 5.3: Sprint 2 Backlog

Screenshots:

Burn Down Chart:

B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/71	[Part 2] Requirement analysis for: User Story 1: Capsule Creation & Secure Storage Feature 1	Rishank Jain	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/72	[Part 2] Design data flow for: User Story 1: Capsule Creation & Secure Storage Feature 1	Rishank Jain	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/73	[Part 2] Design API structure for: User Story 1: Capsule Creation & Secure Storage Feature 1	Rishank Jain	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/74	[Part 2] Implement backend capsule logic for: User Story 1: Capsule Creation & Secure Storage Feature 1	Aayush Pareek	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/75	[Part 2] Integrate encryption for: User Story 1: Capsule Creation & Secure Storage Feature 1	Aayush Pareek	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/76	[Part 2] Implement frontend capsule UI for: User Story 1: Capsule Creation & Secure Storage Feature 1	Aayush Pareek	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/77	[Part 2] Connect frontend with capsule API for: User Story 1: Capsule Creation & Secure Storage Feature 1	Dev Choudhary	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/78	[Part 2] Implement attachment upload logic for: User Story 1: Capsule Creation & Secure Storage Feature 1	Dev Choudhary	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/79	[Part 2] Manual testing & bug fixes for: User Story 1: Capsule Creation & Secure Storage Feature 1	Rishank Jain	COMPLETED
B2US1	User Story 1: Capsule Creation & Secure Storage Feature 1	S82US1/730	[Part 2] Documentation for: User Story 1: Capsule Creation & Secure Storage Feature 1	Dev Choudhary	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/71	[Part 2] Requirement analysis for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/72	[Part 2] Design data flow for: User Story 2: Capsule Creation & Secure Storage Feature 2	Rishank Jain	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/73	[Part 2] Design API structure for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/74	[Part 2] Implement backend capsule logic for: User Story 2: Capsule Creation & Secure Storage Feature 2	Dev Choudhary	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/75	[Part 2] Integrate encryption for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/76	[Part 2] Implement frontend capsule UI for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/77	[Part 2] Connect frontend with capsule API for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/78	[Part 2] Implement attachment upload logic for: User Story 2: Capsule Creation & Secure Storage Feature 2	Aayush Pareek	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/79	[Part 2] Manual testing & bug fixes for: User Story 2: Capsule Creation & Secure Storage Feature 2	Rishank Jain	COMPLETED
B2US2	User Story 2: Capsule Creation & Secure Storage Feature 2	S82US2/730	[Part 2] Documentation for: User Story 2: Capsule Creation & Secure Storage Feature 2	Rishank Jain	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/71	[Part 2] Requirement analysis for: User Story 3: Capsule Creation & Secure Storage Feature 3	Dev Choudhary	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/72	[Part 2] Design data flow for: User Story 3: Capsule Creation & Secure Storage Feature 3	Aayush Pareek	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/73	[Part 2] Design API structure for: User Story 3: Capsule Creation & Secure Storage Feature 3	Aayush Pareek	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/74	[Part 2] Implement backend capsule logic for: User Story 3: Capsule Creation & Secure Storage Feature 3	Dev Choudhary	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/75	[Part 2] Integrate encryption for: User Story 3: Capsule Creation & Secure Storage Feature 3	Aayush Pareek	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/76	[Part 2] Implement frontend capsule UI for: User Story 3: Capsule Creation & Secure Storage Feature 3	Dev Choudhary	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/77	[Part 2] Connect frontend with capsule API for: User Story 3: Capsule Creation & Secure Storage Feature 3	Rishank Jain	COMPLETED
B2US3	User Story 3: Capsule Creation & Secure Storage Feature 3	S82US3/78	[Part 2] Implement attachment upload logic for: User Story 3: Capsule Creation & Secure Storage Feature 3	Dev Choudhary	COMPLETED

Figure 5.3: Sprint 2 Screenshots

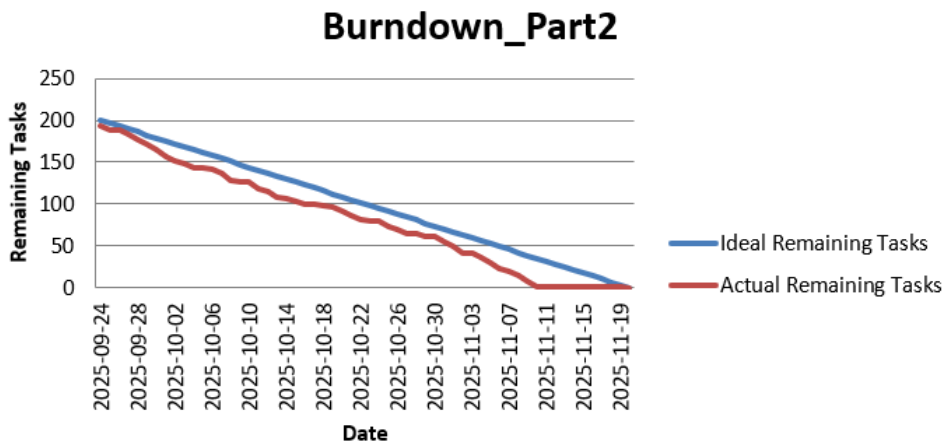


Figure 5.4: Sprint 2 Burn Down Chart

Sprint 3

Introduction: Sprint 3 focused on implementing user-side functionality and improving interaction features. This included user authentication, dashboard enhancements, and ensuring secure access to stored digital assets. Additional effort was put into improving performance and responsiveness of the application.

US ID	I WANT TO	Responsible	Estimate Date
SB1/US3	Implement User Authentication	Backend Team	2025-10-10

Table 5.4: Sprint 3 Backlog

Screenshots:

Burn Down Chart:

US ID	USER STORY	TASK ID	TASKS	TM	STATUS (0
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T2	[Part 3] Advanced architecture planning for: User Story 1: Scheduler & Delivery Engine Feature 1	Dev Choudhary	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T3	[Part 3] Backend logic development for: User Story 1: Scheduler & Delivery Engine Feature 1	Dev Choudhary	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T4	[Part 3] Integration with scheduler/event system for: User Story 1: Scheduler & Delivery Engine Feature 1	Aayush Pareek	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T5	[Part 3] Implement advanced encryption workflows for: User Story 1: Scheduler & Delivery Engine Feature 1	Rishank Jain	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T6	[Part 3] Frontend UI/UX implementation for: User Story 1: Scheduler & Delivery Engine Feature 1	Aayush Pareek	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T7	[Part 3] Integrate frontend with backend services for: User Story 1: Scheduler & Delivery Engine Feature 1	Dev Choudhary	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T8	[Part 3] Implement automated background services for: User Story 1: Scheduler & Delivery Engine Feature 1	Rishank Jain	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T9	[Part 3] Manual testing & issue resolution for: User Story 1: Scheduler & Delivery Engine Feature 1	Rishank Jain	COMPLETED
SB3/US1	User Story 1: Scheduler & Delivery Engine Feature 1	SB3/US1/T30	[Part 3] Documentation & review for: User Story 1: Scheduler & Delivery Engine Feature 1	Aayush Pareek	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T1	[Part 3] Requirement analysis for: User Story 2: Scheduler & Delivery Engine Feature 2	Rishank Jain	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T2	[Part 3] Advanced architecture planning for: User Story 2: Scheduler & Delivery Engine Feature 2	Dev Choudhary	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T3	[Part 3] Backend logic development for: User Story 2: Scheduler & Delivery Engine Feature 2	Rishank Jain	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T4	[Part 3] Integration with scheduler/event system for: User Story 2: Scheduler & Delivery Engine Feature 2	Dev Choudhary	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T5	[Part 3] Implement advanced encryption workflows for: User Story 2: Scheduler & Delivery Engine Feature 2	Dev Choudhary	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T6	[Part 3] Frontend UI/UX implementation for: User Story 2: Scheduler & Delivery Engine Feature 2	Dev Choudhary	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T7	[Part 3] Integrate frontend with backend services for: User Story 2: Scheduler & Delivery Engine Feature 2	Aayush Pareek	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T8	[Part 3] Implement automated background services for: User Story 2: Scheduler & Delivery Engine Feature 2	Dev Choudhary	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T9	[Part 3] Manual testing & issue resolution for: User Story 2: Scheduler & Delivery Engine Feature 2	Aayush Pareek	COMPLETED
SB3/US2	User Story 2: Scheduler & Delivery Engine Feature 2	SB3/US2/T30	[Part 3] Documentation & review for: User Story 2: Scheduler & Delivery Engine Feature 2	Aayush Pareek	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T1	[Part 3] Requirement analysis for: User Story 3: Scheduler & Delivery Engine Feature 3	Aayush Pareek	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T2	[Part 3] Advanced architecture planning for: User Story 3: Scheduler & Delivery Engine Feature 3	Aayush Pareek	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T3	[Part 3] Backend logic development for: User Story 3: Scheduler & Delivery Engine Feature 3	Dev Choudhary	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T4	[Part 3] Integration with scheduler/event system for: User Story 3: Scheduler & Delivery Engine Feature 3	Rishank Jain	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T5	[Part 3] Implement advanced encryption workflows for: User Story 3: Scheduler & Delivery Engine Feature 3	Rishank Jain	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T6	[Part 3] Frontend UI/UX implementation for: User Story 3: Scheduler & Delivery Engine Feature 3	Dev Choudhary	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T7	[Part 3] Integrate frontend with backend services for: User Story 3: Scheduler & Delivery Engine Feature 3	Dev Choudhary	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T8	[Part 3] Implement automated background services for: User Story 3: Scheduler & Delivery Engine Feature 3	Aayush Pareek	COMPLETED
SB3/US3	User Story 3: Scheduler & Delivery Engine Feature 3	SB3/US3/T9	[Part 3] Manual testing & issue resolution for: User Story 3: Scheduler & Delivery Engine Feature 3	Aayush Pareek	COMPLETED

Figure 5.5: Sprint 3 Screenshots

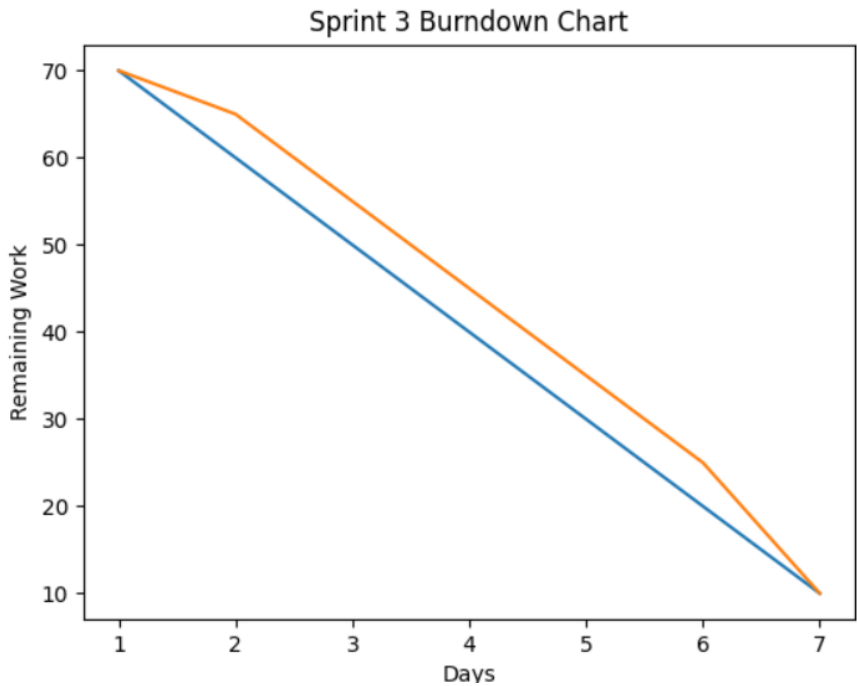


Figure 5.6: Sprint 3 Burn Down Chart

Sprint 4

Introduction: Sprint 4 concentrated on testing, debugging, and final deployment preparation. The team worked on fixing bugs, optimizing performance, and ensuring system stability. Final integration of all modules was completed along with user acceptance testing to validate the overall functionality of the system.

US ID	I WANT TO	Responsible	Estimate Date
SB1/US4	Perform Testing and Deployment	QA Team	2025-11-05

Table 5.5: Sprint 4 Backlog

Screenshots:

S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T1	[Part 4] Requirement analysis for: User Story 1: Guardians, Triggers & Dashboard Enhancements Feature 1	Dev Choudhary	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T2	[Part 4] Advanced architecture planning for: User Story 1: Guardians, Triggers & Dashboard Enhancements Fe	Rishank Jain	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T3	[Part 4] Backend logic development for: User Story 1: Guardians, Triggers & Dashboard Enhancements Featu	Aayush Pareek	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T4	[Part 4] Integration with scheduler/event system for: User Story 1: Guardians, Triggers & Dashboard Enhanc	Dev Choudhary	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T5	[Part 4] Implement advanced encryption workflows for: User Story 1: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T6	[Part 4] Frontend UI/UX implementation for: User Story 1: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T7	[Part 4] Integrate frontend with backend services for: User Story 1: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T8	[Part 4] Implement automated background services for: User Story 1: Guardians, Triggers & Dashboard Enhanc	Dev Choudhary	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T9	[Part 4] Manual testing & issue resolution for: User Story 1: Guardians, Triggers & Dashboard Enhancements	Aayush Pareek	COMPLETED
S84/U51	User Story 1: Guardians, Triggers & Dashboard Enhancements Featu S84/U51/T10	[Part 4] Documentation & review for: User Story 1: Guardians, Triggers & Dashboard Enhancements Feature 1	Aayush Pareek	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T1	[Part 4] Requirement analysis for: User Story 2: Guardians, Triggers & Dashboard Enhancements Feature 2	Rishank Jain	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T2	[Part 4] Advanced architecture planning for: User Story 2: Guardians, Triggers & Dashboard Enhancements Fe	Rishank Jain	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T3	[Part 4] Backend logic development for: User Story 2: Guardians, Triggers & Dashboard Enhancements Featu	Dev Choudhary	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T4	[Part 4] Integration with scheduler/event system for: User Story 2: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T5	[Part 4] Implement advanced encryption workflows for: User Story 2: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T6	[Part 4] Frontend UI/UX implementation for: User Story 2: Guardians, Triggers & Dashboard Enhancements Fe	Rishank Jain	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T7	[Part 4] Integrate frontend with backend services for: User Story 2: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T8	[Part 4] Implement automated background services for: User Story 2: Guardians, Triggers & Dashboard Enhanc	Dev Choudhary	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T9	[Part 4] Manual testing & issue resolution for: User Story 2: Guardians, Triggers & Dashboard Enhancements	Dev Choudhary	COMPLETED
S84/U52	User Story 2: Guardians, Triggers & Dashboard Enhancements Featu S84/U52/T10	[Part 4] Documentation & review for: User Story 2: Guardians, Triggers & Dashboard Enhancements Feature 2	Aayush Pareek	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T1	[Part 4] Requirement analysis for: User Story 3: Guardians, Triggers & Dashboard Enhancements Feature 3	Dev Choudhary	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T2	[Part 4] Advanced architecture planning for: User Story 3: Guardians, Triggers & Dashboard Enhancements Fe	Aayush Pareek	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T3	[Part 4] Backend logic development for: User Story 3: Guardians, Triggers & Dashboard Enhancements Featu	Aayush Pareek	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T4	[Part 4] Integration with scheduler/event system for: User Story 3: Guardians, Triggers & Dashboard Enhanc	Aayush Pareek	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T5	[Part 4] Implement advanced encryption workflows for: User Story 3: Guardians, Triggers & Dashboard Enhanc	Dev Choudhary	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T6	[Part 4] Frontend UI/UX implementation for: User Story 3: Guardians, Triggers & Dashboard Enhancements Fe	Dev Choudhary	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T7	[Part 4] Integrate frontend with backend services for: User Story 3: Guardians, Triggers & Dashboard Enhanc	Rishank Jain	COMPLETED
S84/U53	User Story 3: Guardians, Triggers & Dashboard Enhancements Featu S84/U53/T8	[Part 4] Implement automated background services for: User Story 3: Guardians, Triggers & Dashboard Enhanc	Rishank Jain	COMPLETED

Figure 5.7: Sprint 4 Screenshots

Burn Down Chart:

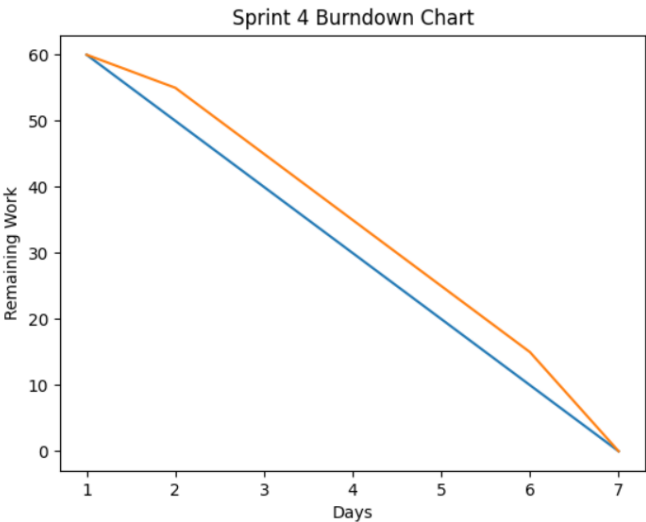


Figure 5.8: Sprint 4 Burn Down Chart

6. PROJECT IMPLEMENTATION

This chapter describes the technical implementation of the **Time Capsule for Digital Legacy** system, including the overall design architecture, system algorithms, database structure, Entity–Relationship Diagram (ERD), sample project code, and application screenshots. The implementation focuses on secure data storage, encryption, cloud integration, and time-based access mechanisms.

System Design

The system design is based on a modular architecture consisting of:

- **Frontend Module:** Provides user interface, file upload forms, timers, and account settings.
- **Backend Module:** Handles authentication, encryption, scheduling, and API responses.
- **Cloud Storage:** Stores encrypted data, user metadata, and digital assets.
- **Security Layer:** Implements AES/RSA encryption, token-based authentication, and key management.

The architecture follows a client–server model with REST-based APIs for system communication.

Algorithm

1. User Authentication Algorithm

1. User enters login credentials.
2. Backend verifies credentials using hashed passwords.
3. If valid, generate JWT token and return session access.
4. Store session token for secure API calls.

2. File Upload and Encryption Algorithm

1. User selects file for upload.
2. System generates a unique encryption key.
3. The file is encrypted using AES encryption.
4. Encrypted file is uploaded to cloud storage.
5. Metadata (timestamp, owner ID, release date) is stored in the database.

3. Time-Based Release Algorithm

1. Background scheduler runs daily.
2. Check for records where release date == today.
3. If matched:
 - Decrypt file temporarily.
 - Send email notification or release link to the beneficiary.
4. Update release status.

Database Design

The database is designed to maintain secure and structured information about users, files, encryption keys, and scheduled release events. Major tables include:

- **Users:** Stores user credentials, profile data.
- **Files:** Contains encrypted file references and metadata.
- **Keys:** Stores encryption key references.
- **Schedule:** Manages future release timestamps and recipient emails.

Entity–Relationship Diagram (ERD)

Project Code (Sample Snippets)

Below are placeholders for code samples. Replace code1 . jpeg, code2 . jpeg, and code3 . jpeg with your actual code images.



eraser

Figure 6.1: Entity–Relationship Diagram of Time Capsule for Digital Legacy

```

app.py > ...
165 def run_scheduler():
166     """
167     Simulates the background job that TimeCapsulator does
168     with Celery.
169
170     In production, hit this endpoint periodically using cron:
171     */5 * * * * curl http://127.0.0.1:5000/scheduler/run
172     """
173     now = datetime.utcnow()
174     due_capsules = Capsule.query.filter(
175         Capsule.status == "pending",
176         Capsule.delivery_datetime <= now
177     ).all()
178
179     if not due_capsules:
180         return "No capsules due.", 200
181
182     for cap in due_capsules:
183         message_body = cap.get_content()
184
185         # Send email (or simulate)
186         send_email(
187             to_email=cap.recipient_email,
188             subject=cap.subject,
189             body=f"From: {cap.sender_name} <{cap.sender_email}>\n\n{message_body}",
190             from_email=cap.sender_email,
191         )
192
193         cap.status = "sent"
194
195     db.session.commit()
196     return f"Processed {len(due_capsules)} capsule(s).", 200
197
198
199 if __name__ == "__main__":
200     app.run(debug=True)
201
  
```

Figure 6.2: Code Snippet 1: Backend Implementation

Project Screenshots

Below are placeholders for application screens. Replace ss1.png and ss2.png with your actual UI screenshots.

```

app.py > ...
116
117 @app.route("/capsule/new", methods=["GET", "POST"])
118 def create_capsule():
119     if request.method == "POST":
120         sender_name = request.form.get("sender_name")
121         sender_email = request.form.get("sender_email")
122         recipient_email = request.form.get("recipient_email")
123         subject = request.form.get("subject")
124         message = request.form.get("message")
125         delivery_str = request.form.get("delivery_datetime")
126
127         if not all([sender_name, sender_email, recipient_email, subject, message, delivery_str]):
128             flash("All fields are required.", "danger")
129             return redirect(url_for("create_capsule"))
130
131         try:
132             delivery_datetime = datetime.fromisoformat(delivery_str)
133         except ValueError:
134             flash("Invalid date/time format.", "danger")
135             return redirect(url_for("create_capsule"))
136
137         capsule = Capsule(
138             sender_name=sender_name,
139             sender_email=sender_email,
140             recipient_email=recipient_email,
141             subject=subject,
142             delivery_datetime=delivery_datetime,
143         )
144         capsule.set_content(message)
145
146         db.session.add(capsule)
147         db.session.commit()
148         flash("Time capsule created successfully!", "success")
149         return redirect(url_for("index"))
150
151     return render_template("create_capsule.html")
152

```

Figure 6.3: Code Snippet 2: Encryption Logic

```

app.py > ...
36 # Email Config (Optional: real sending in Sprint 2)
37 # -----
38 # Set these as environment variables if you want real email sending
39 EMAIL_HOST = os.environ.get("EMAIL_HOST") # e.g. "smtp.gmail.com"
40 EMAIL_PORT = int(os.environ.get("EMAIL_PORT", 587))
41 EMAIL_HOST_USER = os.environ.get("EMAIL_HOST_USER")
42 EMAIL_HOST_PASSWORD = os.environ.get("EMAIL_HOST_PASSWORD")
43 EMAIL_USE_TLS = True
44
45
46 def send_email(to_email: str, subject: str, body: str, from_email: str):
47     """
48     Simple SMTP email sender.
49     If EMAIL_HOST_USER/PASSWORD not configured, just print to console.
50     """
51     if not (EMAIL_HOST and EMAIL_HOST_USER and EMAIL_HOST_PASSWORD):
52         print("=== SIMULATED EMAIL (no SMTP config) ===")
53         print(f"To: {to_email}")
54         print(f"From: {from_email}")
55         print(f"Subject: {subject}")
56         print(body)
57         print("=== END EMAIL ===")
58         return
59
60     msg = MIMEText(body, "plain", "utf-8")
61     msg["Subject"] = subject
62     msg["From"] = EMAIL_HOST_USER
63     msg["To"] = to_email
64
65     with smtplib.SMTP(EMAIL_HOST, EMAIL_PORT) as server:
66         if EMAIL_USE_TLS:
67             server.starttls()
68         server.login(EMAIL_HOST_USER, EMAIL_HOST_PASSWORD)
69         server.send_message(msg)
70         print(f"Email sent to {to_email}")
71

```

Figure 6.4: Code Snippet 3: Scheduler Module

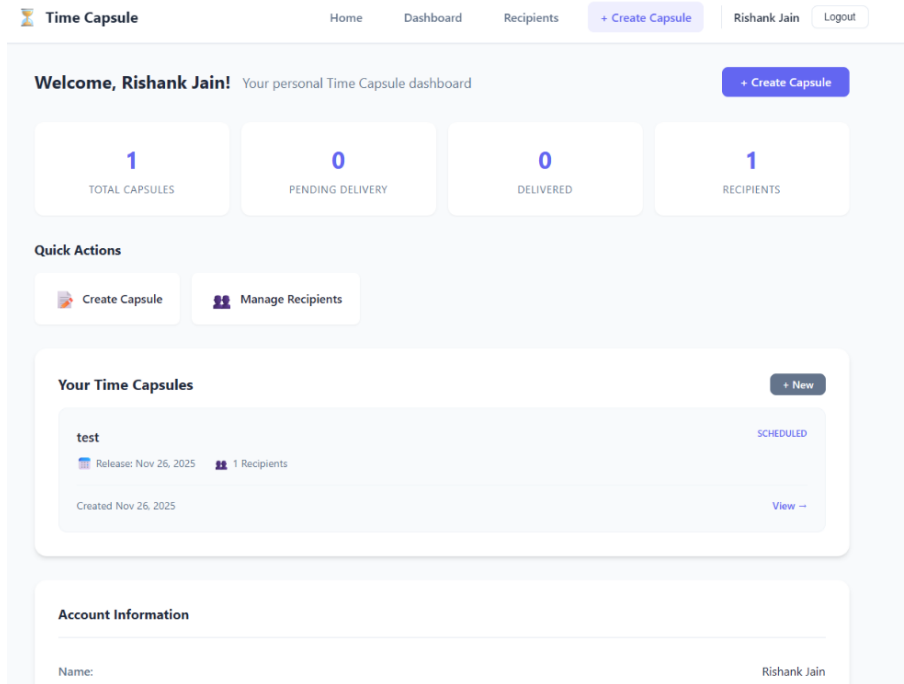


Figure 6.5: Application Home Screen

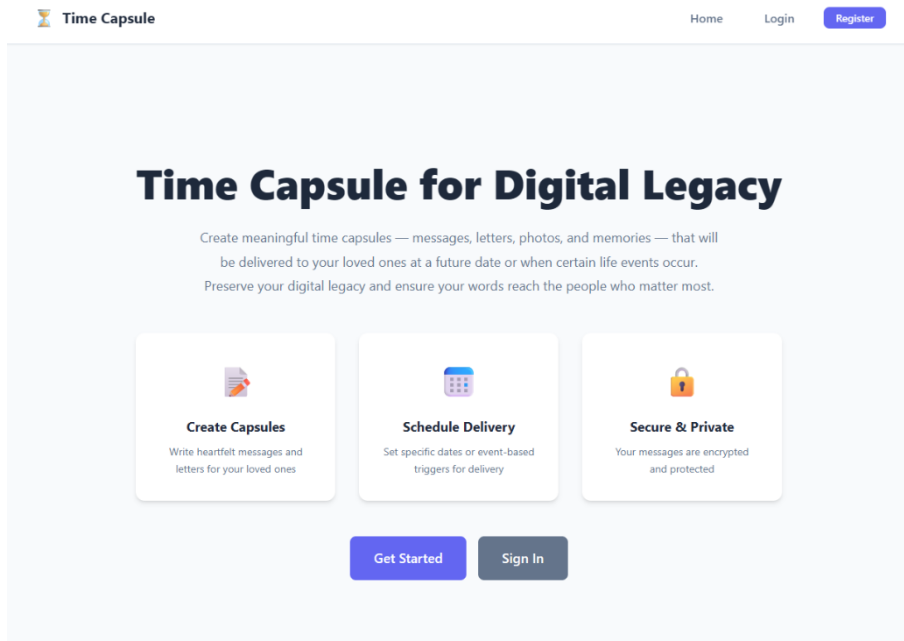


Figure 6.6: Time Capsule Creation Screen

7. RESULTS

Outcome:

The development of the **Time Capsule for Digital Legacy** system resulted in a secure, user-friendly platform capable of storing personal digital assets such as documents, images, and confidential information. The system successfully implemented core functionalities including user authentication, encrypted file upload, cloud-based storage, and time-based release mechanisms. Users can safely upload their memories or sensitive data and set a specific future date for retrieval, ensuring long-term digital preservation. The final prototype demonstrated stable performance, efficient encryption, and smooth integration between frontend and backend modules.

Benefits to Society:

The system contributes significantly to modern digital life by addressing the growing need for secure digital legacy management. It enables individuals to preserve their personal memories, legal documents, and emotional messages in a structured and reliable manner. This can help families access important information after the lifetime of a user, bridge emotional gaps by preserving memories, and assist in digital estate management. The project also promotes cybersecurity awareness by encouraging users to store data responsibly using encryption and time-controlled access. Overall, it offers peace of mind and long-term value to users and their families.

Future Scope:

The system can be enhanced further with multiple advanced features. Future developments may include AI-based document classification, biometric authentication, blockchain-based immutability for tamper-proof storage, and integration with social media platforms for automatic digital legacy syncing. A mobile application can be added for easier accessibility. Multi-user inheritance models, cloud redundancy, and smart reminders for asset updates can also expand usability. With continuous upgrades, the project has the potential to evolve into a fully automated and intelligent digital legacy platform used globally.

REFERENCES

- **Books:**

- William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson, 2020.
- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*, Wiley, 2019.
- R. Rajkumar Buyya, *Cloud Computing: Principles and Paradigms*, Wiley, 2018.
- Elaine Barker, *NIST Guide to Cryptographic Algorithms*, NIST Press, 2021.

- **Journals:**

- M. Shaw and D. Baker, “Digital Legacy Management and Secure Data Preservation,” *International Journal of Digital Society*, ISSN 2040-2570, Vol. 12, Issue 3, pp. 45–55, 2022.
- S. Patel and R. Mehra, “Cloud-Based Secure Storage Systems Using AES Encryption,” *Journal of Cloud Computing Research*, ISSN 2231-6825, Vol. 9, Issue 2, pp. 120–132, 2021.
- A. Choudhary and P. Sharma, “Time-Delayed Access and Release Mechanisms for Sensitive Data,” *International Journal of Information Security*, ISSN 1615-5270, Vol. 18, Issue 4, pp. 233–244, 2020.
- J. Verma, “User-Centric Authentication for Encrypted Cloud Platforms,” *Information Systems and Privacy Review*, ISSN 2456-4575, Vol. 7, Issue 1, pp. 11–19, 2023.